InfoVis Design Principles

InfoVis: interactive visual representation of abstract data to amplify cognition

- Purpose: insight, not pictures
- Best for: EDA (**Exploratory Data Analysis**)
- 当不知道要研究什么时用 EDA
- **Two aspects**: representation / interaction
- **Two challenges**: scale / diversity

Edward Tufte's Principles

- **Graphical Integrity**: tell the truth (Check: baseline / scale / context) Avoid distortion and ambiguity of data in charts. (图应与数据正比例；数据标清楚) 不能展示同个数据的差分图。
  Zero Tolerance: 所有柱状图都应当有 0 基准线，否则导致 distortion and exaggeration。比如点图就不一定基准在 0（因为视觉上不比较长短）
  Lie Factor = size of effect in graph / size of effect in data，超过 **1.05** 的图就算显著变形。
  当使用面积而不是长短时（比如圆形），要保证面积而不是直径与数据正比例
- **Tufte Data-Ink Ratio**: DIR = Data_Ink / Total_Ink, need to MAXIMIZE.
  方法: erase non-data-ink / erase redundant data-ink / revise & edit。
  有些图 (尤其是柱状图) 需要具体数据、数据多或对比不明显，用表格反而更好
- **Avoid Chart junk**: 3D / unnecessary color，junk 比如：恒纵轴、图中横线、方框、数据标注过于密集、不必要的形状 (比如箱形图)
- **Data Density =** Number of Entries / Area of Graph
- **Small multiples**: same design, another variable changes
- **Macro/Micro**: big picture + fine details
- **Content is King**

Gestalt Laws

- proximity：人在归类时，空间接近性的优先性高于颜色和图形
- similarity
- connectedness
- continuity：人类的注意力能跟随 ”部分断开的路径“，将其视为连续
- closure：人脑会自动补充不完整图片/图形
- symmetry：Glide-reflectional 旋转对称；translational 平移对称
- relative-size：越大的物体会被判定为越近/越重要，可能造成误解
- figure-ground
- Prägnanz：Less is More

Visual Perception

Perception

- active / contextual, influenced by values, experience and background

Two-stage model:

- Early stage: Low-level, Parallel (color, shape…) 直觉性的自动迅速思考;
- Later stage: Sequential, Top-down, Goal-Directed (pattern, meaning…) 缓慢且需要长期记忆，可分为 2 subsystems: verbal, motor

Three-stage model:

- **Selection**: 先注意什么; **Organization**: 怎么分组; **Interpretation**: 如何理解

Preattentive Processing

不需要集中注意力、通常在 **200–250 ms** 内完成的快速 parallel 视觉处理

- Tasks: target detection / boundary detection / counting
- Single feature 才可以 preattentive
- **Conjuction** of features (like shape and hue) 就需要 Sequential Search
- **Key properties**: color / form / movement / spatial position
- Change Blindness: 有时人对视觉变化无法感知

Color Scheme

即 **0.4~0.7** 微米之间的光。专业者可分别 1M 种，普通人 20,000 种。

- RGB (3 cone cells) 常用于显示屏；CMYK (cyan, magenta, yellow, key-black) 常用于出版物、实体商品
- **HSL**: Hue, Saturation, Lightness 白度
  L=0 黑色, L=100 白色, **L=50 且 S=100 正常纯色**
- **HSV**: Hue, Saturation, Value
  V=0 黑色, **V=100 正常纯色**, S=0 白色
- **Luminance**: foreground/background 要有亮度差
- **Quantitative color**: avoid rainbow scale for quantitative data
  Ware's suggestion: **12 colors**
  Less is more: **more than 20~30 colors brings harm**.
  建议：蓝色可以大面积用（不能太饱和，否则会有残影），不要用很接近的蓝色，红和绿只在视野中心用（人对其不敏感），可以用同个 H 的不同 SL。

D3.js and HTML

网页的标准结构：

1. HyperText Markup Language (HTML) —— 网页的渲染语言
2. Document Object Model (DOM) —— HTML 写的易操作网页结构
3. Cascading Style Sheets (CSS) —— 视觉风格呈现 (颜色、字体、大小、布局…)
4. Scalable Vector Graphics (SVG) —— 矢量绘图系统，不会失真
5. JS-D3 —— 核心：存储/处理数据，实现动态化…

CSS

- Internal CSS： 在 HTML 文件的 `<head> <style>` 区域原地编辑 h1, p, body 等组件的 color, font…
- External CSS：在 HTML 文件引用链接 `<link rel="stylesheet" href="xxx.css">`

JS

- **console.log()**：打印
- **object.key 等价于 object[“key”]**：对象属性可用 **dot notation** 或 **square bracket notation** 访问。
  ⚠ 坑人：当属性名以特殊字符或数字开头，或要用 **variable as key** 时只能用方框。
- 单双引号不敏感、分号不敏感
- 所有没有标明 var 或 let 的声明产生的变量都是 **global**
- `[x, y, z]` 代表数组，`{a: b, c: d}` 代表字典
- **this**: 在对象方法里，通常指当前对象。
- **const**: constant reference, not constant value
  ⚠ 坑人：**can** change array elements / object properties; but **cannot** reassign the whole array/object reference (改字段可以，整个改不行)

JS Arrays

`[]`：数组本质上也是 object。

- 对象靠字符 key，数组靠数字 index，从 0 开始。
- **push()**: 加入数组尾部
- **pop()**: 返回最后一项并删除它
- **shift()**: 返回第一项并删除它
- **unshift()**: 加入一个或多个元素到数组头上并返回数组长度
- **slice(start, end)**: 浅拷贝 `[start, end)`
  ⚠ 返回数组一部分的**浅拷贝**，原数组不变。
- **map()**: 通过某个函数映射所有元素到新数组，返回这个新数组 `map1 = arr.map(x => x + 2)`
  ⚠ **When not to use map()**:not using the returned array; or not returning a value from callback
- **forEach()**: calls a function once for each array element
  ⚠ 和 map 不同，它重点是“遍历执行”，不是生成新数组。

D3

JavaScript library, data-driven DOM / HTML / SVG manipulation

- **Selection**: based on CSS selectors
  select() first matching; **selectAll()** all matching
  select by ID ("#xxx"), by className (".xxx"), by tag ("div", "h1", "p", "span", etc.)
  The return value of most selection methods is the selection itself. This means that selection methods such as .style, .attr and .on can be **chain**ed. E.g.: `selection.attr(...).style(...).text(...)`
- **append()** 追加元素 child
- **text()** 设置文字
- 改 HTML 内容用什么？→ **html()**
- 改属性用什么？→ **attr()**
- 改 CSS 样式用什么？→ **style()**

SVG

scalable vector graphics `<svg>` : SVG 图形的画布。Create with**: d3.select("#vis").append("svg")

- **circle**: cx, cy, r. 可选：**fill**, **stroke**, **stroke-width**. **Create with**: .append("circle")
- **rect**: x, y, width, height (后两者不能为负)
- **line**: 起点 x1, y1; 终点 x2, y2
- path: **Capital letters**: absolute position；**Lowercase letters**: relative position
- **text**: x, y, text-anchor
- **g**: vis.append("g") 然后再 group.append("circle"), group.append("rect"), group.append("text")。
- 显示元素的更新、控制：
  d = datum; **i =** index. **Use i for position (where), Use d for size / style (what/how)**
  ⚠ **data join** 四步骤 d3.select() → .selectAll() → .data() → .join()
  数据一旦变化，通常需要重新做 join，这样以后 update(newData) 就能重画 / 更新
  .join(enter, update, exit) 三个函数分别处理有数据时元素的进入和更新；没有数据时元素的退出。
- 元素的变换：
  translate(tx, ty): move along x and y
  rotate(angle): rotate around origin or specified center
  scale(sx, sy): resize, if sy omitted then sy = sx
  Linear scale: domain → range
  scaleSqrt: circle area encoding 控制面积而不是直径与数据成比例
  axis: axisBottom / Left / Right / Top + .call() (生成 + 调用)
  transition: state change animation

Multivariate Data & Tables

Data

- Variables of data are attributes (特性) or dimensions (维度)
- **Data model**: often described by **Objects 对象**, **Attributes 属性** and **Relations 关系**
- Data table：**Case =** 行里那个“个体“，**Variable =** 列里那个“属性” Wide table: each attribute gets a column (e.g. year) Long table: each data case has an attribute-value pair，更整洁 (便于作图) Variable Type:
  i. **Categorial**: **Norminal** (不可排序：如 sex, country_tag) or **Ordinal**(可排序：如 clothing_size)
  ii. **Quantitative** (可数的 Numeric)
  Metadata 元数据: 描述数据的数据 (such as yaml/toml header)
- 数据编码和提高空间利用率的方案：Composition, allignment, folding, recursion, overloading。

Data Issues and Cleaning：

- Data Issue: **Missing fields**, **Duplicates**, **Conflicts**, **Delimiters 反斜杠**, white spaces, encoding, abbrev., typos, syntax ……..
- Data cleaning: **remove, impute (guess) or flag (e.g. NAN) missing ones**. Only remove an outlier if you can prove that it’s **erroneous**.
- Cleaning steps: Remove unwanted or irrelevant observations, Fix structural errors, Standardization (单位、格式、大小写), Remove unwanted outliers, Fix contradiction, Type conversion and syntax errors, Deal with missing data, Validate dataset.

Multivariate Data, Table and Graph:

- **Univariate** data graph: x-axis: independent cases, y-axis: dependent variables
- **Bivariate**: Scatter-point is most often
- **Trivariate** and **Hypervariate** scatter graph: **use small multiples, or colors in 2D**
- **Table** = exact values. 数字通常右对齐，并按小数位数对齐，更利于比较。
- **Graph** = patterns / shape / relationships。

Multivariate Graphs & Charts

Few's process: Choose chart (or table) based on data type and analytic goal, not decoration.

Basic Plots:

Scatter plot: 适合展示 **2** 个 quantitative variables 的关系，对多变量关系不佳。

- 如果非规则时间间隔，scatterplot 可能比 line chart 更合适。
  Line chart: 用于连续时间上的变化趋势。
- 只要不会误导，纵坐标可以不从 0 开始。
- **Banking to 45°**
- 如果太多重合线 (Overplotting)，用 **Small Multiples**。
- 当数值跨越不同数量级，而你想比较 **growth rate**（比如都增长 10%），**Log scale** 比 Linear scale 好。
  Bar chart: 因为比较的是 **length**，纵坐标不从 0 开始会严重夸大差异。
- **Horizontal bars** 在 **标签太长** 、 **case 太多** 或者 **柱太长** 的情况下比竖直柱状图好。
- **Stacked bars**: 可以在一个 bar 里叠加分类组成。
  Trellis display: 本质就是 small multiples。比如 Stacked Bar 信息过多可采用
  Crosstab: 适合类别变量之间的多变量 (超过 2x2) 交叉分析。
  Pareto chart: 本质是 **bar chart + line chart** 的组合。
- ⚠ **80/20 Rule**: 80% of effect comes from 20% of causes (**Vital few**)。
- 适合分析问题或缺陷的根源，处理投诉/销量/事故分析
  Bump chart: 展示排序关系随时间变动。

Distribution Analysis Plots:

Box plot: 箱型图用于看分布、离散程度、可能的异常值。
Histogram: 直方图用于连续数据，柱子相连，展示分布。⚠ 柱状图则是离散数据、柱子不相连。

- 多个 dataset 叠加时可能 cluttered—— **可以用 line chart 解决**。
  Dot plot: 解释一些数据在数据集中出现的频率 (用点表示，类似于用点的直方图)
- ⚠ 主要是单变量或少数组别比较（1维为主），Scatter plot 是成对双变量（2维）。
  Strip plot: 一种有轴带类别的散点图，display individual observations for each category along a single axis。
  Stem-and-leaf plot: 茎叶图 display shape and spread of a continuous distribution，可看 **central tendency, variability, skewness, outliers**。

Cyclical Data Plots:

Radar graph: represent data across the cyclical nature of time

- 适合带周期性的时间结构。
  Heatmap: display a large quantity of cyclical data
- ⚠ 当周期性数据太多，不适合 radar 时可用 heatmap。

Overplotting: 数据点重合，难度。

- 解决方案：Reduce size, **Jitter points**,Make points transparent, Aggregate or sample data, **Encode density**。

Graph theory and graph visualization

Graph theory

图由节点 (实体) 和边 (关系) 构成，$\boldsymbol{G}$=($\boldsymbol{V}$,$\boldsymbol{E}$)。具体图论请点击此处。
图的两种表达方式：

- **Adjacency Matrix**：随机访问，O(|$\boldsymbol{V}$|2)，矩阵中 (a, b) 为 1 表示 ab 边存在；

- **Adjacency List**：顺序访问，O(|$\boldsymbol{E}$|)，用列表 a: b 表示 ab 边存在。

  节点一般用数字 Index，便于 manage and program；通过 map (Java), dictionary (Python) 或者 SQLite 分配数字 ID。
  ⚠ 图可以帮助 Business: Detect risk patterns, simulate internet/railway/supply chain etc., maximize asset mix in market, map social hierarchy, detect communities……

- 图不一定是简单的 (Multiple Edges)，比如 *LinkedIn* 中的人际关系。

Graph in Visualization

图的应用和对应种类 (**Graph Layouts**)：

- (Node-link) **Relationships** → nodes + links, conceptual picture

- **Hierarchies** → trees, sunburst 日冕图

- **Communities** → clustered layouts, labels, computational clustering

- **Flows** → Sankey, chord diagram

- **Spatial networks** → schematic layout for routes/nodes

- **Large graphs** → overplotting or tile-based aggregation

- **Process of Visualization Graphs** (作图过程)：
  1. Prepare Data
  2. Layout 编排点和边
  3. Add Visual Attributes 大小颜色位置、标注边

4. ⚠ Interact, Explain, Further Analysis：Zoom and pan, Identify (Hover / Click), **Filter** (⚠ 不是删除，只是隐藏), Isolate, Neighbors, Paths, **Drag-move-modify-delete-group**, Explanation Sequence (e.g. *Gephi*), Annotate, **Legend** (图例), Export.

- ⚠ 2 *Gephi* Filters：**Giant component 拓扑的去除小或离散的点**，**Edge weight 去掉出现频率低的点。**
- **ego-network:** 某个中心点及其所有邻居 (focal node + its direct neighbors / relationships)。

图的基础知识：

- ⚠ **Basic graph statistics**：**Density**, **Components**, **Degree**, **Path**, **Centrality**. (Hue 不算)

  Centrality：**Betweeness** Centrality (经过最短路径的个数，越大越中心), **Katz** Centrality (距离所有点的反比例加权和的大小), **Degree** Centrality (度数，越大越中心), **Closeness** Centrality (到所有点的平均距离)。

  Diameter：最长的 “最短路径” (即 Critical Path)。

- ⚠ **Essential visual attributes**: Node size, Node color, Label, Edge weight, Edge color, Edge type.

编排 (**Layouts** 方案)：

页面编排：

- **Random node layouts**：空间中随机分布，比如 **Force Atlas 2 layout**
- **force-directed layout**：一个巨大的聚簇以及零星分散的点

图 (本身的) 编排：

- Force-directed (力导向), Node-only (只有点), Time-oriented (按时间排序的树图), Top-down (从上到下从打到小), Radial (辐射状), Treemap (nested-rectangle 方块面积表示阶级), Hierarchical Pie (**圆形 hierarchy，和 sunburst 很像**), Maps , Chord, Parallel Coordinates (与 Bipartite 交互)。
- ⚠ Hierarchy Layout 包括了 **tree 非圆形树图** 和 **radial 发散图**。其中 tree 表示非圆形的整齐排列，包括了 top-down 和 left-to-right，top-down 因为太宽容易写不下 label。如果阶级很宽 (点很多) 更适合用 radial 。radial 包括了 radial-tree (普通的树状发散图) 和 sunburst chart (类似于 pie)。
- Sankey diagram 适合有方向、步骤、路径的 flow (比如产品供应流向)；而 Chord diagram 适合多对多无既定关系的 flow (比如全球进出口)

图的 Encoding：**Node =** entity，**Link =** relationship，**Size** often encodes importance / popularity，**Color** often encodes frequency / attribute，**Width** often encodes link amount / strength。

- Communities Vis：**Clique** 即 “完全子图” (内部两两相连)，用颜色区分不同主要的 Cliques。

  Convex hull 即包裹每个组 (community) 周围的 ”壳“，适合突出 group，而非细看每条 link。

- Flow Vis：用于display transactional events, depict systems / behavioral models, supply chain.

  ⚠ **Transactional data 通常用 Sankey Diagram**。**Sankey =** multi-step flow + link **width encodes volume**。